

Platform Amendment — Version 4.10.0

Vermont Health Platform (HTR)

Type: Amendment to Platform Documentation (supplements v4.9.0 and all prior amendments) **Version:** 4.10.0
Date: April 2026 **Classification:** Internal **Scope:** Three architectural areas that were previously undocumented: (1) the HomeSidebar navigation system — structure, state management, and visual hierarchy; (2) the six-pillar design system — color registry, route registry, and the rules for adding pillars; (3) the RightSidebar AI Analyst layout — the exact CSS constraints that must never be changed and why they exist.

Table of Contents

- 1. [HomeSidebar Navigation Architecture](#)
- 2. [Six Pillars Design System](#)
- 3. [RightSidebar AI Analyst — Layout Constraints](#)

1. HomeSidebar Navigation Architecture

1.1 — Philosophy: Sidebar-First Navigation

The platform follows a **sidebar-first** navigation principle: all navigation lives in the left sidebar. Pages show only content — no navigation cards, no in-page "go to section X" buttons, no hub-page tabs that duplicate sidebar items.

This is an intentional design decision, not a default. When considering whether to put navigation in a page, the answer is always: put it in the sidebar.

1.2 — File

`frontend/components/HomeSidebar.tsx`

1.3 — Section Structure

The sidebar has **five collapsible L1 sections** plus a standalone My Library link.

Section ID	Label	Header color	Typical routes
<code>learn</code>	Learn	sky	<code>/academy/*</code>
<code>analyze</code>	Analyze & Tools	amber	<code>/research-lab/*</code> , <code>/htr-simulator</code> , <code>/hti-dashboard</code> , <code>tools</code>
<code>intelligence</code>	Intelligence	slate	<code>/policy/*</code> , <code>/economics/*</code> , <code>/technology/*</code> , <code>/clinical/*</code> , <code>/equity/*</code> , <code>/operations/*</code>
<code>states</code>	States & Programs	rose	<code>/vermont-act-167</code> , <code>/california-calaim</code> , <code>/states</code> , <code>/dashboard</code> , <code>/ahead-model</code>
<code>advisory</code>	Advisory & Services	indigo	<code>/advisory/*</code> , <code>/connect-hub</code> , <code>/connect/*</code> , <code>/community</code>
<code>(standalone)</code>	My Library	slate	<code>/saved</code>

The **Intelligence** section is special: instead of a flat item list, it renders the six pillar accordions (L2), each of which expands to reveal sub-items (L3).

1.4 — State Management

State is managed with two `useState` arrays inside `HomeSidebar`:

```
const [expandedSections, setExpandedSections] = useState<string[]>([]);
const [expandedPillars, setExpandedPillars] = useState<string[]>([]);
```

`expandedSections` — which L1 sections are open. Multiple sections can be open simultaneously. The user must explicitly close a section by clicking — it never auto-collapses.

`expandedPillars` — which Intelligence pillars are open within the Intelligence section. Same multi-open, user-only-close behavior.

Auto-open on navigation

A `useEffect` watching `pathname` automatically opens the relevant section and pillar when the user navigates to a new route (including via the address bar or a non-sidebar link):

```
useEffect(() => {
  const section = getSectionForPath(pathname);
  if (section && !expandedSections.includes(section)) {
    setExpandedSections((prev) => [...prev, section]);
  }
  const pillar = getPillarForPath(pathname);
  if (pillar && !expandedPillars.includes(pillar)) {
    setExpandedPillars((prev) => [...prev, pillar]);
  }
}, [pathname]);
```

Note: the effect only *adds* to the expanded arrays — it never removes. This ensures a user who has manually opened sections doesn't find them collapsed just because they navigated away.

Route-to-section mapping

`getSectionForPath(pathname)` — returns the section ID that should be open for a given route:

```
function getSectionForPath(path: string): string | null {
  const intelligencePrefixes = ["/policy", "/economics", "/technology",
"/clinical", "/equity", "/operations"];
  if (intelligencePrefixes.some((p) => path === p || path.startsWith(p + "/")))
return "intelligence";
  if (path === "/academy" || path.startsWith("/academy/")) return "learn";
  // ...etc
}
```

`getPillarForPath(pathname)` — returns the pillar ID for Intelligence sub-routes.

Rule: When adding a new route, update both helper functions if the route belongs to an existing section, or add a new section entry if it is a new top-level area.

1.5 — Visual Hierarchy

The sidebar has three levels of visual depth:

L1 — Section headers (always visible)

- Button, full-width, no border
- Closed: no background, `text-slate-600`
- Open: `${section.headerBg}` (e.g. `bg-sky-100`), `${section.headerColor}` (e.g. `text-sky-700`)
- Icon: colored `w-7 h-7 rounded-lg` badge wrapping a Heroicon

- Label: `text-[12px] font-medium tracking-wide`

L2 — Section items / pillar headers (visible when section is open)

- Links or buttons inside a `border-l-2 border-slate-200` rail
- Active item: `bg-${color}-100`
- Hover: `hover:bg-${color}-100`
- Font: `text-[13px] font-medium`
- Pillar headers have an additional chevron icon that rotates when open

L3 — Pillar sub-items (visible when pillar is open within Intelligence)

- Inside a `border-l-2 ${pillar.rail}` colored rail (e.g. `border-l-sky-400` for Policy)
- Font: `text-[11px] font-normal text-slate-500`
- Active: `text-${pillar.accent}` (e.g. `text-sky-700`) + `font-semibold`

No descriptions on any item. Every sidebar item is label + optional icon only. No subtitle text, no hover descriptions. This applies without exception.

1.6 — Active State

`isActive(href)` uses **exact match only**:

```
const isActive = (href: string) => pathname === href;
```

There is no parent-route highlighting. `/policy` is not highlighted when the user is at `/policy/regulation`. Only the exact matching link gets the active style.

1.7 — Adding a New Item to an Existing Section

1. Add the item to the `items` array of the relevant section in `SECTIONS` inside `HomeSidebar.tsx`
2. If the route is not already covered by `getSectionForPath`, add a prefix to the appropriate section's prefix array
3. Add the same route to `Header.tsx` (mega-menu) and `app/sitemap.ts`

1.8 — Adding a New L1 Section

1. Add a new entry to `SECTIONS` in `HomeSidebar.tsx` with a unique `id`, color tokens, and `items` array
2. Add the section's route prefixes to `getSectionForPath`
3. Update `Header.tsx` if this section should also appear in the desktop mega-menu
4. Update this document

2. Six Pillars Design System

2.1 — The Pillars

The platform's intelligence content is organized into **six pillars**. Each pillar has a unique color identity used consistently across the sidebar, pillar hub pages, onboarding modal, and all pillar-specific components.

#	Pillar	ID	Tailwind color	Primary route
1	Policy	<code>policy</code>	<code>sky</code>	<code>/policy</code>
2	Economics	<code>economics</code>	<code>emerald</code>	<code>/economics</code>
3	Technology	<code>technology</code>	<code>indigo</code>	<code>/technology</code>
4	Clinical	<code>clinical</code>	<code>red</code>	<code>/clinical</code>
5	Equity	<code>equity</code>	<code>violet</code>	<code>/equity</code>
6	Operations	<code>operations</code>	<code>teal</code>	<code>/operations</code>

2.2 — Per-Pillar Token Set

Each pillar entry in the `pillars` array in `HomeSidebar.tsx` carries a consistent set of tokens:

```
{
  id:    "policy",
  label: "Policy",
  href:  "/policy",
  dot:   "bg-sky-500",      // filled circle indicator
  accent:"text-sky-700",    // active/selected text color
  rail:  "border-l-sky-400", // L3 colored left border rail
}
```

When adding a pillar:

1. Choose a Tailwind color that is not already used by another pillar
2. Use `-500` for `dot`, `-700` for `accent`, `-400` for `rail` — consistent across all six
3. Add the same color to `PillarHub.tsx` (the shared pillar hub template), `Header.tsx` (mega-menu), and any pillar-specific hub page

2.3 — Where the Color Appears

Location	Usage
<code>HomeSidebar.tsx</code> — <code>pillars</code> array	<code>dot</code> , <code>accent</code> , <code>rail</code> tokens
<code>HomeSidebar.tsx</code> — L3 sub-items	<code>\${pillar.rail}</code> left border
<code>components/templates/PillarHub.tsx</code>	Hub page header accent, badge text color
<code>app/[pillar]/page.tsx</code> each pillar	Header badge, CTA button, card hover
<code>components/OnboardingModal.tsx</code>	Pillar list in welcome screen
<code>components/Header.tsx</code>	Mega-menu pillar color dot
<code>components/LatestHubReports.tsx</code>	<code>colorClass</code> prop passed from pillar pages
<code>components/HubSubscribeCTA.tsx</code>	<code>bgClass</code> and <code>buttonClass</code> props

2.4 — Pillar Hub Page Template

All six top-level pillar pages use `PillarHub.tsx` as a shared template:

```
// frontend/components/templates/PillarHub.tsx
interface PillarHubProps {
  pillar: string;      // e.g. "Policy"
  colorClass: string;  // e.g. "text-sky-700"
  badgeClass: string;  // e.g. "bg-sky-100 text-sky-700"
  cardHoverClass: string; // e.g. "hover:border-sky-400 hover:bg-sky-50/80"
  titleHoverClass: string; // e.g. "group-hover:text-sky-700"
  tagline: string;
  topics: PillarTopic[];
}
```

Pass all color props from the individual pillar page. Do not hardcode colors inside the template.

2.5 — The Sixth Pillar (Operations)

Operations was the sixth pillar added (April 2026). It uses `teal` — the `bg-teal-500` / `text-teal-700` / `border-l-teal-400` set. It has its own full page at `/operations` with five sub-items. The onboarding modal, developer guide §8, and all references to "five pillars" were updated to "six pillars" at the same time.

If you see any remaining reference to "five pillars" anywhere in the codebase or documentation, it is a bug — update it.

2.6 — Adding a Seventh Pillar

Before adding a new pillar:

1. Choose a Tailwind color not already used by the six above

2. Create `/app/[pillar-id]/page.tsx` using `PillarHub.tsx`

3. Add the pillar entry to `pillars` array in `HomeSidebar.tsx`

4. Add to `Header.tsx` mega-menu

5. Add to `getPillarForPath()` in `HomeSidebar.tsx`

6. Add to `OnboardingModal.tsx` pillar list

7. Update `app/sitemap.ts`

8. Update this document and write a platform amendment

3. RightSidebar AI Analyst — Layout Constraints

3.1 — Background

The right sidebar contains the AI Analyst chat interface. Getting the layout right required significant debugging effort. The working solution is fragile in the sense that changing any of the specific CSS properties listed below causes the input box to either be buried below the viewport or overflow the sidebar container. **Do not change any of these properties without fully understanding the constraint and testing the result across viewport sizes.**

3.2 — Architecture Overview

The AI Analyst has two modes:

Mode	Location	Entry	Exit
Compact widget	Right sidebar, always available	Click "Ask AI" floating button or sidebar toggle	Click expand icon → goes to <code>/chat</code>
Full chat	<code>/app/chat/page.tsx</code>	Click <code>ArrowsPointingOutIcon</code> in sidebar header	Click <code>ArrowsPointingInIcon</code> → calls <code>setRightOpen(true)</code> then <code>router.back()</code>

When `pathname === "/chat"`, `AppShell` forces both sidebars closed and hides them. The collapse button on the full chat page pre-sets `setRightOpen(true)` via `SidebarContext` before navigating back, so the right sidebar reopens on the previous page.

A floating "Ask AI" button appears fixed at bottom-right only when the right sidebar is closed.

3.3 — The Working CSS — DO NOT CHANGE

These are the exact properties that must be maintained across three files:

`frontend/components/CollapsibleSidebar.tsx`

```
const SIDEBAR_BOTTOM_GAP = "7rem";
```

The sidebar wrapper style:

```
style={{
  top: stickyTop,
  minHeight: "30vh",
  maxHeight: `calc(100vh - ${stickyTop} - ${SIDEBAR_BOTTOM_GAP})`,
}}
```

Why `minHeight` + `maxHeight` instead of fixed `height`: The sidebar grows from `30vh` up to nearly the full viewport before the messages scroll area activates. A fixed height would prevent this growth at smaller viewport heights. `SIDEBAR_BOTTOM_GAP = 7rem` ensures the input box is not cut off by the bottom of the viewport.

Inner div (inside the aside):

```
<div className="w-[85vw] md:w-72 h-full flex flex-col relative">
```

`h-full` here is **required**. Removing it causes the flexbox height calculation to collapse and the input sinks below the visible area.

`fillHeight` branch (used by the right sidebar):

```
<div className="flex-1 min-h-0 flex flex-col overflow-hidden px-2 pt-3 pb-4">
  {children}
</div>
```

frontend/app/AppShell.tsx

The right `CollapsibleSidebar` must have `fillHeight={true}`:

```
<CollapsibleSidebar
  side="right"
  fillHeight={true}
  {/* ...other props */}
>
  <RightSidebar />
</CollapsibleSidebar>
```

If `fillHeight` is `false` or omitted on the right sidebar, the layout switches to a scrollable container (designed for the left sidebar's nav list), which breaks the AI chat flex layout.

frontend/components/RightSidebar.tsx

The layout chain inside `RightSidebar`:

```
<aside className="w-full h-full flex flex-col">                                {/* ← MUST be h-full */}
  <div className="flex-1 min-h-0 ... flex flex-col overflow-hidden">
    <div>                                                                    {/* header / model selector */}
    </div>
    <div className="flex-1 min-h-0 ... overflow-y-auto"> {/* messages scroll area */}
    </div>
    <div className="shrink-0 ...">                                           {/* ← MUST be shrink-0 */}
      {/* input box */}
    </div>
  </div>
</aside>
```

`h-full` on `<aside>` — without this, the aside does not fill its `CollapsibleSidebar` container and the flex chain breaks.

shrink-0 on the input wrapper — without this, the input box compresses to zero height when the messages area fills the available space. The **flex-1 min-h-0** on the messages area already allows it to shrink; the input must be told explicitly not to.

3.4 — Summary: The Four Properties You Must Not Remove

File	Property	What breaks if removed
<code>CollapsibleSidebar.tsx</code>	<code>minHeight: "30vh" + maxHeight: calc(...)</code> instead of fixed <code>height</code>	Sidebar collapses or clips
<code>CollapsibleSidebar.tsx</code>	<code>SIDEBAR_BOTTOM_GAP = "7rem"</code>	Input box cut off at bottom of viewport
<code>CollapsibleSidebar.tsx</code> inner div	<code>h-full</code>	Flex height collapses, input sinks below viewport
<code>AppShell.tsx</code> right sidebar	<code>fillHeight={true}</code>	Right sidebar switches to scroll container, chat layout breaks
<code>RightSidebar.tsx</code> aside	<code>h-full</code>	Aside doesn't fill container, flex chain collapses
<code>RightSidebar.tsx</code> input wrapper	<code>shrink-0</code>	Input compresses to zero height under message scroll pressure

3.5 — Full Chat Page — `/chat`

`frontend/app/chat/page.tsx` is a standalone full-screen experience. Key behaviors:

- `AppShell` detects `pathname === "/chat"` and hides both sidebars
- Header has two action buttons: **Save** (downloads chat as `.txt`) and **Clear** — both disabled when messages array is empty
- The collapse button (`ArrowsPointingInIcon`) does two things in sequence: `setRightOpen(true)` via `SidebarContext`, then `router.back()`. This ordering ensures the right sidebar is open when the previous page renders
- The floating "Ask AI" button (`fixed bottom-6 right-6`) is suppressed on the `/chat` page (it only renders when the right sidebar is closed AND the current route is not `/chat`)